Smart Chef & Pantry AI

Document 15 — Testing Strategy & Quality Assurance Specification

1. Dokümanın Amacı

Bu dokümanın amacı Smart Chef & Pantry AI uygulamasının Flutter frontend, Flask backend, PostgreSQL database ve recommendation/matching katmanlarının test edilmesi için kullanılacak stratejiyi tanımlamaktır.

Test süreci yalnızca uygulamanın açılıp açılmadığını kontrol etmekten ibaret değildir. Sistem içerisindeki:

- Business logic
- Recipe matching
- Recommendation
- Authentication
- Authorization
- Favorites
- Cart
- Ingredient management
- API validation
- Database işlemleri
- Error handling
- Flutter state management
- Navigation
- UI davranışları

ayrı ayrı test edilmelidir.

Amaç, uygulamanın MVP kapsamındaki temel fonksiyonlarının güvenilir, tutarlı ve tekrar test edilebilir olmasını sağlamaktır.

---

2. Test Prensipleri

Projede aşağıdaki temel prensipler uygulanacaktır:

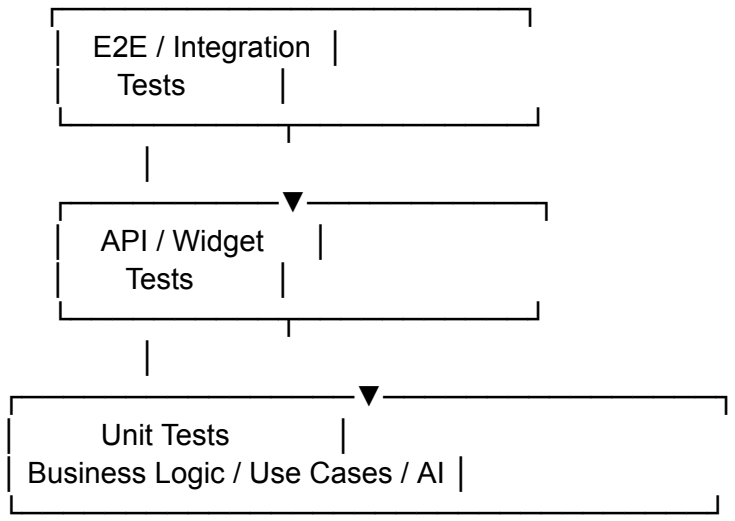
1. Business logic UI'dan bağımsız test edilmelidir.
2. Backend business logic doğrudan unit test edilmelidir.
3. API endpoint'leri integration testlerle doğrulanmalıdır.

4. Database davranışları gerçek PostgreSQL'e mümkün olduğunca yakın ortamda test edilmelidir.
5. Authentication ve authorization ayrı ayrı test edilmelidir.
6. Negatif senaryolar pozitif senaryolar kadar önemlidir.
7. Aynı test tekrar çalıştırıldığında aynı sonucu üretmelidir.
8. Testler birbirinden bağımsız olmalıdır.
9. Kritik business rules için regression testleri oluşturulmalıdır.
10. UI testi yalnızca görünümü değil kullanıcı davranışını da doğrulamalıdır.

---

3. Test Katmanları

Smart Chef & Pantry AI için test piramidi aşağıdaki şekilde olacaktır:



Temel olarak:

Unit Tests
↓
Integration Tests
↓
Widget Tests
↓
End-to-End Tests

şeklinde ilerlenmelidir.

Unit test sayısı en yüksek seviyede tutulmalıdır.

---

4. Test Türleri

Projede aşağıdaki test türleri kullanılacaktır:

4.1. Unit Test

Tek bir sınıf, fonksiyon veya business logic parçasının test edilmesidir.

Örnek:

RecipeMatcher
IngredientNormalizer
CartService
FavoriteService
UseCase
Validator

4.2. Integration Test

Birden fazla component'in birlikte çalışması test edilir.

Örneğin:

API
+
Service
+
Repository
+
PostgreSQL

4.3. Widget Test

Flutter içerisindeki widget ve ekran davranışlarının test edilmesidir.

Örneğin:

FavoriteButton
QuantitySelector
RecipeCard
CartItemCard

4.4. End-to-End Test

Gerçek kullanıcı akışının baştan sona test edilmesidir.

Örneğin:

Login
→ Ingredient Selection
→ Recipe Recommendation
→ Recipe Detail
→ Add Favorite
→ Add Missing Ingredient
→ Cart

4.5. Regression Test

Daha önce çalışan bir özelliğin yeni geliştirmelerden sonra bozulup bozulmadığını kontrol eder.

---

5. Backend Unit Testing

Backend unit testleri özellikle service ve AI katmanlarına odaklanacaktır.

Test edilecek temel sınıflar:

AuthService
IngredientService
RecipeService
FavoriteService
CartService

IngredientNormalizer
RecipeMatcher
RecommendationEngine

---

6. IngredientNormalizer Testleri

IngredientNormalizer aşağıdaki işlemleri gerçekleştirmelidir:

trim
lowercase
whitespace normalization

Örnek:

" DOMATES "

sonucu:

"domates"

olmalıdır.

Birden fazla boşluk:

"domates   sosu"

sonucu:

"domates sosu"

olmalıdır.

Türkçe karakterler korunmalıdır.

Örneğin:

"ÇİLEK"

normalization sonrasında:

"çilek"

olmalıdır.

---

7. Semantic Variant Testleri

Normalizer aşağıdaki dönüşümü otomatik olarak yapmamalıdır:

domates
↓
çeri domates

veya:

domates sosu
↓
domates

Bunlar farklı ingredient kayıtları olarak değerlendirilebilir.

Bu davranışın test edilmesi önemlidir.

Çünkü basit string normalization ile semantic normalization birbirine karıştırılmamalıdır.

---

8. RecipeMatcher Unit Tests

RecipeMatcher projenin kritik business logic bileşenlerinden biridir.

Aşağıdaki testler zorunludur.

Test 1 — Tüm malzemeler mevcut

Recipe:

Chicken
Tomato
Onion

User:

Chicken
Tomato
Onion

Beklenen:

match_percentage = 100
missing = []

Test 2 — Malzemelerin yarısı mevcut

Recipe:

Chicken
Tomato

Onion
Pepper

User:

Chicken
Tomato

Beklenen:

match_percentage = 50

Missing:

Onion
Pepper

Test 3 — Hiçbir malzeme mevcut değil

User:

Milk
Rice

Recipe:

Chicken
Tomato
Onion

Beklenen:

match_percentage = 0

Test 4 — Optional ingredient

Recipe:

Chicken required
Tomato required
Parsley optional

User:

Chicken
Tomato

Beklenen:

match_percentage = 100

Parsley eksik olmasına rağmen yüzde düşmemelidir.

Test 5 — Duplicate user ingredients

User:

Tomato
Tomato
Onion

Recipe:

Tomato
Onion

Duplicate ingredient ikinci kez sayılmamalıdır.

Beklenen:

match_percentage = 100

---

9. RecommendationEngine Testleri

RecommendationEngine:

Normalize
→ Match
→ Filter
→ Rank

akışını yürütür.

Örneğin:

Recipe A = 100%
Recipe B = 75%
Recipe C = 50%
Recipe D = 20%

ve:

MIN_MATCH_PERCENTAGE = 30

ise Recipe D sonuçlardan çıkarılmalıdır.

Sonuçlar:

A
B
C

olmalıdır.

---

10. Recommendation Ranking Tests

Aynı match percentage değerine sahip tariflerde ikinci sıralama kriteri:

missing required ingredients ASC

olmalıdır.

Örneğin:

Recipe A
Match = 75%
Missing = 1

Recipe B
Match = 75%
Missing = 2

Bu durumda sıralama:

Recipe A
Recipe B

olmalıdır.

Aynı değerler mevcutsa sonraki ranking kriterleri uygulanmalıdır.

---

11. Empty Recommendation Test

Request:

```
{
  "ingredient_ids": []
}
```

gönderildiğinde recommendation çalıştırılmamalıdır.

Beklenen:

422

Error:

EMPTY_INGREDIENT_LIST

---

12. Invalid Ingredient Test

Request içerisinde database'de bulunmayan ingredient ID gönderilirse:

INGREDIENT_NOT_FOUND

döndürülmelidir.

Bu durumda recommendation işlemi kısmen gerçekleştirilmemelidir.

---

13. Recipe Service Tests

RecipeService aşağıdaki senaryoları test etmelidir:

Recipe exists
Recipe does not exist
Inactive recipe
Recipe detail
Recipe recommendation
Invalid ingredient
No matching recipes

Inactive recipe kullanıcıya normal recommendation sonucu olarak gönderilmemelidir.

---

14. Favorite Service Tests

FavoriteService testleri:

Add Favorite

Valid user + valid recipe
→ success

Duplicate Favorite

Existing favorite
→ FAVORITE_ALREADY_EXISTS

Remove Favorite

Existing favorite
→ success

Remove Non-existing Favorite

No favorite
→ FAVORITE_NOT_FOUND

Unauthorized Ownership

Bir kullanıcının başka kullanıcıya ait favorite kaydını silmeye çalışması engellenmelidir.

---

15. Cart Service Tests

CartService kritik testlerden biridir.

Test 1 — New Item

Tomato × 2

eklenir.

Beklenen:

Cart = Tomato × 2

Test 2 — Same Item Merge

Mevcut:

Tomato × 2

Yeni:

Tomato × 3

Beklenen:

Tomato × 5

Test 3 — Different Unit

Mevcut:

Tomato × 2 adet

Yeni:

Tomato × 500 gram

Bunlar otomatik olarak:

Tomato × 502

haline getirilmemelidir.

Ayrı cart item olabilir.

Test 4 — Invalid Quantity

quantity = 0

ve:

quantity = -1

reddedilmelidir.

Test 5 — Remove Item

Mevcut cart item silindiğinde database'den kaldırılmalıdır.

Test 6 — Clear Cart

Clear Cart sonrasında kullanıcının cart item sayısı:

0

olmalıdır.

---

16. Authentication Tests

Authentication testleri:

Valid Google token
Invalid Google token
Expired token
Wrong audience
Wrong issuer
Missing token
Malformed JWT
Expired JWT

senaryolarını kapsamalıdır.

---

17. Authorization Tests

Authentication:

"Sen kimsin?"

Authorization:

"Bu işlemi yapmaya yetkin var mı?"

mantığıyla ayrılmalıdır.

Testler:

Guest → protected endpoint
User → own resource
User → another user's resource
Admin → admin endpoint
Normal User → admin endpoint

şeklinde hazırlanmalıdır.

---

18. Admin Authorization Tests

Normal kullanıcı:

POST /api/admin/recipes

gönderirse:

403 Forbidden

almalıdır.

Error:

ADMIN_REQUIRED

olmalıdır.

Admin kullanıcı aynı endpoint'e erişebilmelidir.

---

19. API Integration Tests

API testlerinde gerçek endpoint davranışı kontrol edilir.

Temel endpoint grupları:

/auth
/ingredients
/recipes
/favorites
/cart
/admin

test edilmelidir.

---

20. Authentication API Tests

POST /api/auth/social

Test:

Valid provider
Valid ID token

Beklenen:

200
JWT
User information

Invalid token:

401
SOCIAL_AUTH_FAILED

Invalid provider:

422
INVALID_PROVIDER

---

21. Ingredient API Tests

GET /api/ingredients

testleri:

All ingredients
Search
Category filter
Pagination
Empty result
Invalid page
Invalid limit

kapsamalıdır.

---

22. Recipe API Tests

GET /api/recipes/{recipe_id}

testleri:

Valid recipe
Invalid UUID
Non-existing recipe
Inactive recipe

kapsamalıdır.

Recommendation:

POST /api/recipes/recommend

testleri:

Valid ingredients
Empty ingredients
Invalid ingredient
No results
Threshold filtering
Ranking

kapsamalıdır.

---

23. Favorites API Tests

GET /api/favorites
POST /api/favorites
DELETE /api/favorites/{recipe_id}

testleri:

Authenticated user
Guest user
Valid recipe
Invalid recipe
Duplicate favorite
Remove existing
Remove non-existing
Ownership

senaryolarını içermelidir.

---

24. Cart API Tests

GET /api/cart
POST /api/cart/items
PATCH /api/cart/items/{id}
DELETE /api/cart/items/{id}
DELETE /api/cart

testleri:

Get empty cart
Add item
Merge item
Different unit
Update quantity
Invalid quantity
Remove item
Clear cart
Ownership
Guest access

senaryolarını içermelidir.

---

25. Database Tests

Database testlerinde özellikle constraint'ler kontrol edilmelidir.

Test edilmesi gerekenler:

Foreign Keys
Unique Constraints
Not Null Constraints
Quantity validation
Recipe-Ingredient relationship
Favorite uniqueness
Cart uniqueness

Özellikle:

UNIQUE(user_id, recipe_id)

ve:

UNIQUE(user_id, ingredient_id, unit)

kuralları database seviyesinde test edilmelidir.

---

26. Transaction Tests

Bir işlem birden fazla database yazma operasyonu içeriyorsa transaction kullanılmalıdır.

Örneğin recipe oluşturma:

Create Recipe
+
Create Recipe Ingredients

işlemlerinden biri başarısız olursa bütün transaction rollback edilmelidir.

Test:

Recipe creation
↓
Ingredient insertion fails
↓
ROLLBACK
↓
Recipe should not remain partially created

---

27. Flutter Unit Tests

Flutter tarafında aşağıdaki bileşenler unit test edilmelidir:

Validators
Use Cases
Repository mapping
Model serialization
Failure mapping

State transformations

Örneğin:

AddCartItemUseCase

repository'nin doğru parametrelerle çağrıldığını kontrol etmelidir.

---

28. Flutter Model Serialization Tests

JSON:

```
{
  "id": "123",
  "name": "Tomato"
}
```

Flutter modeline:

IngredientModel

olarak doğru şekilde parse edilmelidir.

Tersi yönde:

Model
→ JSON

dönüşümü de test edilmelidir.

Bu testler özellikle `json_serializable` kullanıldığı için API contract değişikliklerini yakalamada faydalıdır.

---

29. Riverpod Provider Tests

Provider testleri:

Initial State

Loading
Success
Error
Refresh
Retry

durumlarını kapsamalıdır.

Örneğin CartProvider:

Initial
↓
Loading
↓
Loaded

ve API hatasında:

Loading
↓
Error

durumuna geçmelidir.

---

30. Widget Tests

Temel widget'lar:

RecipeCard
FavoriteButton
CartItemCard
QuantitySelector
IngredientChip
MissingIngredientItem

test edilmelidir.

Örneğin FavoriteButton:

Not Favorite
→ click
→ loading

→ Favorite

ve hata durumunda:

Not Favorite
→ click
→ API Error
→ Not Favorite

olmalıdır.

---

31. Cart Widget Tests

QuantitySelector:

+

butonuna basıldığında quantity artmalıdır.

-

butonuna basıldığında quantity azalmalıdır.

Quantity 1 olduğunda remove davranışı doğru şekilde çalışmalıdır.

Cart item silindiğinde item UI'dan kaldırılmalıdır.

---

32. Empty State Tests

Uygulamadaki boş durumlar test edilmelidir.

Favorites
No favorite recipes yet.

Cart
Your cart is empty.

Recipe Results

No recipes found.

Search

No ingredients found.

Empty state ekranlarının crash olmadan görüntülenmesi gerekir.

---

33. Error State Tests

API hatalarında uygulama kullanıcıya anlamlı mesaj göstermelidir.

Örneğin:

Network Error

durumunda:

Internet connection could not be established.

ve:

Retry

butonu gösterilebilir.

Server:

500

durumunda teknik exception kullanıcıya doğrudan gösterilmemelidir.

Yanlış:

SQLAlchemy IntegrityError at...

Doğru yaklaşım:

Something went wrong. Please try again.

---

34. Navigation Tests

Temel navigation akışları test edilmelidir.

Guest

Home
→ Recipe Detail
→ Favorite
→ Login

Authenticated User

Home
→ Search
→ Recipe Results
→ Recipe Detail
→ Favorites

Cart

Recipe Detail
→ Missing Ingredient
→ Add to Cart
→ Cart

---

35. End-to-End Test Scenario 1

Recipe Recommendation Flow

Open Application
↓
Home
↓
Ingredient Selection
↓
Select Tomato
↓
Select Chicken

↓
Search Recipes
↓
Recipe Results
↓
Recipe Detail

Beklenen:

- Seçilen ingredient'lar doğru gönderilir.
- Recommendation API çağrılır.
- Match percentage görüntülenir.
- Matched ingredients görüntülenir.
- Missing ingredients görüntülenir.

---

36. End-to-End Test Scenario 2

Favorite Flow

Recipe Detail
↓
Favorite
↓
Login
↓
Google Authentication
↓
Return Recipe Detail
↓
Favorite
↓
Favorites Page

Beklenen:

Recipe appears in Favorites.

---

37. End-to-End Test Scenario 3

Missing Ingredients → Cart

Ingredient Selection
↓
Recipe Results
↓
Recipe Detail
↓
Missing Ingredients
↓
Add Missing Ingredients
↓
Cart

Beklenen:

Missing ingredient
→ Cart item

olarak görünmelidir.

Aynı ingredient daha önce sepette varsa miktarı artırılmalıdır.

---

38. End-to-End Test Scenario 4

Cart Management

Open Cart
↓
Increase Quantity
↓
Decrease Quantity
↓
Remove Item
↓
Clear Cart

Her işlem sonrasında backend state ile UI state eşleşmelidir.

---

39. Test Data Strategy

Test verileri production verilerinden ayrı tutulmalıdır.

Önerilen:

Development Database
Test Database
Production Database

ayrımıdır.

Test database içerisinde örnek:

Users
Ingredients
Recipes
Recipe Ingredients
Favorites
Cart Items

seed data bulunabilir.

---

40. Test Fixtures

Backend:

```
tests/
├── fixtures/
│   ├── users.py
│   ├── ingredients.py
│   ├── recipes.py
│   ├── favorites.py
│   └── cart.py
```

oluşturulabilir.

Örnek fixture:

test_user
test_admin
test_recipe

test_ingredients
test_cart

şeklinde kullanılabilir.

Fixture'ların amacı testlerde sürekli aynı database hazırlık kodunun tekrar edilmesini önlemektir.

---

41. Mock Kullanımı

Unit testlerde external dependency'ler mocklanabilir.

Örneğin:

Google Authentication

gerçek Google servisine her unit testte bağlanmamalıdır.

Bunun yerine:

Mock Google Token Validator

kullanılabilir.

Benzer şekilde Flutter'da:

Dio
RemoteDataSource
Repository

mocklanabilir.

Integration testlerde ise gerçek test environment kullanılmalıdır.

---

42. Test Environment

Backend test environment:

FLASK_ENV=test

DATABASE_URL=test_database_url
JWT_SECRET_KEY=test_secret

şeklinde ayrı tutulmalıdır.

Production:

DATABASE_URL
JWT_SECRET_KEY
GOOGLE_CLIENT_ID
GOOGLE_CLIENT_SECRET

gibi gerçek değerler test ortamında kullanılmamalıdır.

---

43. API Contract Testing

Frontend ve backend arasındaki JSON contract test edilmelidir.

Örneğin backend:

```
{
  "match_percentage": 75.0
}
```

gönderiyorsa Flutter modelinin bu alanı doğru parse ettiği doğrulanmalıdır.

Alan adı değişirse:

match_percentage

yerine:

matchPercentage

gibi değişiklikler kontrol edilmeden yapılmamalıdır.

API contract değişiklikleri frontend'i doğrudan etkileyebilir.

---

44. Regression Test Strategy

Her yeni özellik eklendiğinde mevcut kritik testler tekrar çalıştırılmalıdır.

Örneğin Cart'a:

Bulk Add

özelliği eklendiğinde mevcut:

Add Cart Item
Cart Merge
Update Quantity
Remove Item
Clear Cart

testleri de tekrar çalıştırılmalıdır.

Özellikle aşağıdaki alanlar regression test açısından kritiktir:

Authentication
Recipe Matching
Favorites
Cart Merge
Authorization
API Error Handling

---

45. Test Priority

Testler aşağıdaki önceliklere ayrılacaktır.

P0 — Critical

Authentication
Authorization
Recipe Matching
Recommendation
Favorite Add/Remove
Cart Add/Merge
Cart Update/Remove
API validation

Bu testlerden biri başarısızsa MVP'nin ilgili bölümü güvenilir kabul edilmemelidir.

P1 — Important

Pagination
Search
Empty states
Widget interactions
Navigation
Model serialization

P2 — Future

Offline mode
Advanced personalization
LLM ingredient extraction
Bulk cart operations
Market integration
Price comparison

---

46. Minimum MVP Test Matrix

Modül	Unit	Integration	Widget	E2E
Authentication	✓	✓	✓	✓
Ingredients	✓	✓	✓	✓
Recipe Matching	✓	✓	-	✓
Recommendation	✓	✓	✓	✓
Recipe Detail	✓	✓	✓	✓
Favorites	✓	✓	✓	✓
Cart	✓	✓	✓	✓
Error Handling	✓	✓	✓	✓
Admin	✓	✓	-	-

---

47. Critical Business Rules Test Matrix

Business Rule	Test
Duplicate favorite engellenir	✓
Favorite ownership kontrol edilir	✓
Cart aynı ingredient'i merge eder	✓
Farklı unit otomatik birleştirilmez	✓
Quantity 0 veya negatif olamaz	✓
Cart ownership kontrol edilir	✓
Optional ingredient percentage'i düşürmez	✓
Duplicate ingredient bir kez sayılır	✓
Inactive recipe recommendation'a girmez	✓
Empty ingredient list reddedilir	✓
Invalid ingredient ID reddedilir	✓
Admin endpoint normal kullanıcıya kapalıdır	✓
JWT olmayan protected request reddedilir	✓

---

48. Performance Testleri

MVP'nin ilk aşamasında kapsamlı load testing zorunlu değildir.

Ancak temel performans noktaları gözlemlenmelidir.

Özellikle:

GET /api/ingredients
POST /api/recipes/recommend
GET /api/favorites
GET /api/cart

endpoint'leri takip edilmelidir.

Recommendation işlemi recipe sayısı arttığında gereksiz database sorguları oluşturmamalıdır.

N+1 query problemi özellikle:

recipes
+
recipe_ingredients
+
ingredients

ilişkilerinde kontrol edilmelidir.

---

49. Database Query Performance

Aşağıdaki alanlarda index kullanımı ve sorgu performansı kontrol edilmelidir:

ingredients.normalized_name
ingredients.category

recipe_ingredients.recipe_id
recipe_ingredients.ingredient_id

favorites.user_id
favorites.recipe_id

cart_items.user_id
cart_items.ingredient_id

Test database küçük olsa bile production büyüklüğünde veri düşünüldüğünde sorgu yapısı buna göre tasarlanmalıdır.

---

50. Security Testing

Security testleri:

Invalid JWT
Expired JWT
Missing JWT
Forged JWT

Unauthorized resource access
Admin privilege escalation
Invalid UUID
SQL injection attempts
Malformed JSON
Oversized input

gibi durumları kapsamalıdır.

SQL sorguları kullanıcıdan gelen string interpolation ile oluşturulmamalıdır.

SQLAlchemy parametreli sorgular kullanılmalıdır.

---

51. Sensitive Data Tests

Testlerde aşağıdaki bilgilerin loglara yazılmadığı kontrol edilmelidir:

JWT
Google ID Token
JWT Secret
Google Client Secret
Database Password

Örneğin:

Yanlış:

User login token: eyJhbGci...

Doğru:

User authentication successful.

---

52. Error Handling Testleri

Document 14'te tanımlanan error code'lar test edilmelidir.

Özellikle:

AUTH_REQUIRED

INVALID_TOKEN
TOKEN_EXPIRED
SOCIAL_AUTH_FAILED
VALIDATION_ERROR
INVALID_UUID
INVALID_QUANTITY
EMPTY_INGREDIENT_LIST
INVALID_PROVIDER
RECIPE_NOT_FOUND
NO_RECIPES_FOUND
INGREDIENT_NOT_FOUND
FAVORITE_NOT_FOUND
FAVORITE_ALREADY_EXISTS
CART_ITEM_NOT_FOUND
INVALID_CART_QUANTITY
FORBIDDEN
ADMIN_REQUIRED

her biri için en az bir ilgili test senaryosu bulunmalıdır.

---

53. Test Naming Convention

Test isimleri test edilen davranışı açık şekilde ifade etmelidir.

Örneğin:

test_normalizer_trims_whitespace()

test_recipe_matcher_returns_100_when_all_required_ingredients_match()

test_optional_ingredient_does_not_reduce_match_percentage()

test_cart_merges_same_ingredient_and_unit()

test_user_cannot_delete_another_users_cart_item()

Test isimleri teknik olarak neyin test edildiğini anlaşılır şekilde belirtmelidir.

---

54. Test File Structure

Backend:

```
tests/
├── unit/
│   ├── test_auth_service.py
│   ├── test_ingredient_normalizer.py
│   ├── test_recipe_matcher.py
│   ├── test_recommendation_engine.py
│   ├── test_favorite_service.py
│   └── test_cart_service.py
│
├── integration/
│   ├── test_auth_api.py
│   ├── test_ingredient_api.py
│   ├── test_recipe_api.py
│   ├── test_favorite_api.py
│   ├── test_cart_api.py
│   └── test_admin_api.py
│
└── fixtures/
    ├── users.py
    ├── recipes.py
    ├── ingredients.py
    └── cart.py
```

Flutter:

```
test/
├── core/
├── features/
│   ├── auth/
│   ├── ingredients/
│   ├── recipes/
│   ├── favorites/
│   ├── cart/
│   └── profile/
└── integration/
```

---

55. CI/CD Test Strategy

GitHub repository kullanıldığı için ilerleyen aşamada GitHub Actions ile otomatik test çalıştırılması önerilir.

Temel pipeline:

Push / Pull Request
↓
Install Dependencies
↓
Run Backend Tests
↓
Run Flutter Tests
↓
Build / Analyze
↓
Result

Backend testleri başarısızsa pipeline başarısız olmalıdır.

Flutter testleri başarısızsa pipeline başarısız olmalıdır.

---

56. Static Analysis

Flutter:

flutter analyze

ve:

flutter test

çalıştırılmalıdır.

Backend:

pytest

kullanılmalıdır.

Kod kalitesi için ilerleyen aşamada:

Ruff
Black
Flake8

gibi araçlardan uygun olanlar kullanılabilir.

Exact tooling implementation aşamasında proje ihtiyaçlarına göre kesinleştirilecektir.

---

57. Definition of Done

Bir feature tamamlanmış kabul edilmeden önce:

[ ] Business logic yazıldı
[ ] Unit test yazıldı
[ ] API test edildi
[ ] Validation eklendi
[ ] Error handling eklendi
[ ] Authorization kontrol edildi
[ ] Flutter state yönetimi tamamlandı
[ ] Loading state eklendi
[ ] Error state eklendi
[ ] Empty state eklendi
[ ] Widget testleri yazıldı
[ ] İlgili regression testleri çalıştırıldı

kontrol edilmelidir.

---

58. MVP Release Test Checklist

Release öncesinde:

Authentication

[ ] Google login
[ ] Invalid token
[ ] Expired token
[ ] Logout
[ ] Protected endpoint

Ingredients

[ ] Search
[ ] Category
[ ] Pagination
[ ] Selection

Recipes

[ ] Recommendation
[ ] Match percentage
[ ] Matched ingredients
[ ] Missing ingredients
[ ] Recipe detail
[ ] Source URL
[ ] YouTube URL

Favorites

[ ] Add
[ ] Remove
[ ] List
[ ] Duplicate prevention
[ ] Ownership

Cart

[ ] Add
[ ] Merge
[ ] Quantity update
[ ] Remove
[ ] Clear
[ ] Different units
[ ] Ownership

Security

[ ] JWT validation
[ ] Admin authorization
[ ] User ownership
[ ] No secrets in repository
[ ] No sensitive tokens in logs

---

59. Test Coverage Hedefi

MVP için tüm kodun %100 test edilmesi zorunlu değildir.

Öncelik business logic'tedir.

Özellikle:

RecipeMatcher
RecommendationEngine
CartService
FavoriteService
AuthService
Validators

yüksek test coverage'a sahip olmalıdır.

UI'da ise tüm widget'ların satır satır test edilmesinden ziyade kritik kullanıcı akışlarının doğrulanması tercih edilmelidir.

Coverage tek başına kalite göstergesi olarak değerlendirilmemelidir.

Örneğin %90 coverage'a sahip ancak yanlış business rule'u test eden bir sistem güvenilir kabul edilmemelidir.

---

60. Test Sonuçlarının Kaydedilmesi

Test sonuçları development sürecinde takip edilmelidir.

Örneğin:

Test Run #01
Date: 2026-XX-XX

Passed: XX
Failed: XX
Skipped: XX

Critical test failure varsa ilgili feature release öncesinde tekrar incelenmelidir.

---

61. Bug Severity

Bulunan hatalar aşağıdaki şekilde sınıflandırılabilir.

Critical

Uygulamanın temel işlevlerinden birini tamamen engelleyen hata.

Örnek:

Login çalışmıyor.
Recommendation API sürekli crash oluyor.
Database bağlantısı çalışmıyor.

High

Önemli bir özelliğin kullanılamamasına neden olan hata.

Örnek:

Cart item eklenemiyor.
Favorite silinemiyor.

Medium

Feature çalışıyor ancak belirli bir senaryoda yanlış davranıyor.

Örnek:

Empty state yanlış mesaj gösteriyor.

Low

Kullanıcı deneyimini etkileyen ancak sistemi engellemeyen hata.

Örnek:

Küçük UI hizalama problemi.

---

62. Regression Önceliği

Bir hata düzeltildikten sonra yalnızca hatanın bulunduğu test çalıştırılmamalıdır.

Örneğin Cart merge bug'ı düzeltildiyse:

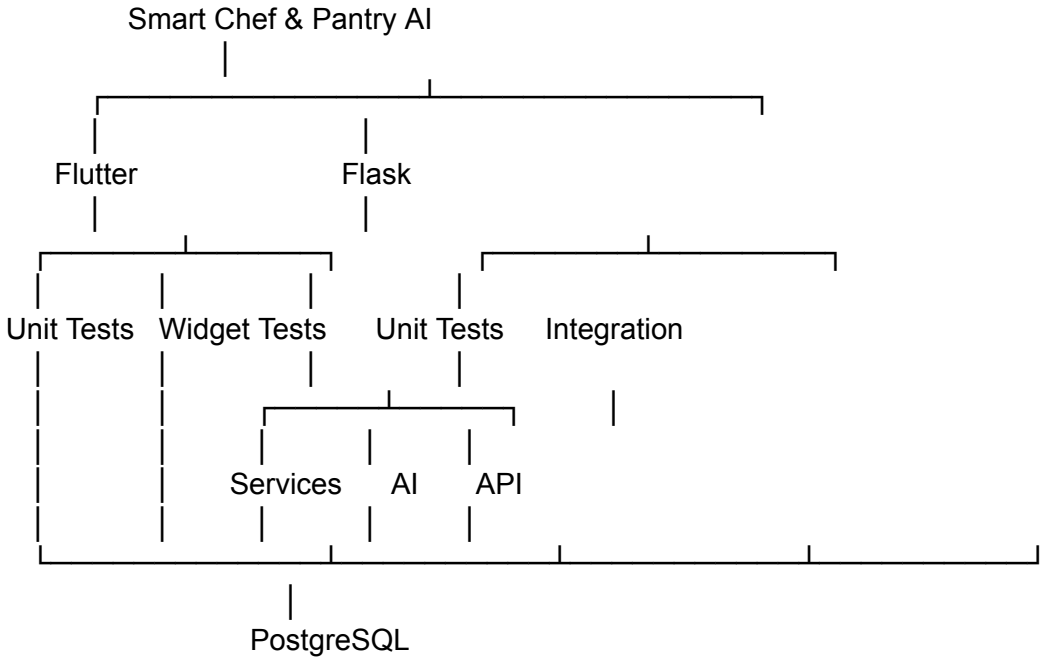
Add Cart Item
Merge
Update
Remove
Clear
Ownership

testleri tekrar çalıştırılmalıdır.

Bu yaklaşım, bir düzeltmenin başka bir Cart davranışını bozmasını engellemeye yardımcı olur.

---

63. Genel Test Mimarisi



---

64. Kritik Test Akışı

Projenin en önemli kullanıcı akışı aşağıdaki şekilde uçtan uca test edilmelidir:

User
↓
Login
↓
Ingredient Selection
↓
Recipe Recommendation
↓
Match Calculation
↓
Recipe Detail

↓
Missing Ingredients
↓
Add to Cart
↓
Cart Merge
↓
Favorite Recipe
↓
Favorites

Bu akış MVP'nin ana değer zincirini temsil etmektedir.

---

65. MVP Test Sonucu Kriteri

MVP'nin test aşamasından geçebilmesi için:

1. Critical authentication testleri başarılı olmalıdır.
2. Authorization testleri başarılı olmalıdır.
3. Recipe matching testleri başarılı olmalıdır.
4. Recommendation testleri başarılı olmalıdır.
5. Favorite uniqueness ve ownership testleri başarılı olmalıdır.
6. Cart merge testleri başarılı olmalıdır.
7. Quantity validation testleri başarılı olmalıdır.
8. API validation testleri başarılı olmalıdır.
9. Kritik Flutter kullanıcı akışları çalışmalıdır.
10. Kritik regression testlerinde başarısızlık bulunmamalıdır.

Bu kriterler sağlanmadan uygulamanın MVP test süreci tamamlanmış kabul edilmemelidir.

---

66. Gelecek Sürümler İçin Test Genişletmeleri

İlerleyen sürümlerde:

Performance Testing
Load Testing
Stress Testing
Security Penetration Testing

Offline Testing
LLM/NLP Evaluation
Recommendation Quality Evaluation
Personalization Testing
Market API Integration Testing

eklenebilir.

Özellikle ML/NLP tabanlı özellikler eklendiğinde klasik unit testlerin yanında model değerlendirme metrikleri de kullanılmalıdır.

Örneğin NLP ingredient extraction için:

Precision
Recall
F1-score

gibi metrikler değerlendirilebilir.

Ancak bu değerlendirmeler MVP'nin deterministic recommendation sistemi için zorunlu değildir.

---

67. Final Teknik Kararlar

Smart Chef & Pantry AI test mimarisinde:

Unit Tests
+
Integration Tests
+
Widget Tests
+
E2E Tests

birlikte kullanılacaktır.

Özellikle:

Recipe Matching
Recommendation
Cart Merge
Favorites
Authentication

Authorization

kritik business logic olarak kabul edilecektir.

Frontend yalnızca UI davranışını test etmekle kalmayacak; backend ile API contract uyumu da doğrulanacaktır.

Backend tarafında service ve AI katmanları yüksek öncelikli unit test kapsamına alınacaktır.

Database constraints yalnızca uygulama koduna bırakılmayacak ve database seviyesinde de doğrulanacaktır.

MVP release öncesinde kritik kullanıcı akışları uçtan uca test edilecektir.

---

68. Sonuç

Bu test stratejisi ile Smart Chef & Pantry AI uygulamasının yalnızca "çalışıyor" olması değil, tanımlanan business kurallarına uygun şekilde ve farklı hata senaryolarında öngörülebilir davranması hedeflenmektedir.

Test mimarisinin temel yaklaşımı:

Kod Yaz
↓
Unit Test
↓
Integration Test
↓
Widget Test
↓
E2E Test
↓
Regression Test
↓
Release

şeklindedir.

Özellikle recommendation algoritması, cart merge sistemi, favorites uniqueness, authentication ve authorization mekanizmaları projenin kritik noktaları olduğundan bu alanlarda test kapsamı yüksek tutulacaktır.

Bu doküman, uygulamanın geliştirme sürecinde yeni özellikler eklendikçe güncellenebilecek yaşayan bir teknik doküman olarak kabul edilir.